

CUADERNO DE LABORATORIO

Laboratorio de Modelos Generativos

Aprender, de lo más básico a lo más avanzado, cómo una red neuronal aprende a crear imágenes — y cómo un bucle de mejora agéntica nos lleva de "funciona regular" a "funciona bien".

Autoencoder

VAE

GAN

Diffusion

INTRODUCCIÓN

¿Qué queremos aprender?

Una escalera de cuatro modelos, del más simple al más capaz

Este cuaderno acompaña a un **laboratorio visual interactivo** creado para *entender* los modelos generativos de imágenes. En lugar de leer fórmulas, se tocan los parámetros, se entrena en vivo y se ve —literalmente— qué aprende cada red. Todo corre en local sobre un conjunto de **43.102 caras de anime de 64x64**, sin etiquetas.

Recorremos cuatro modelos en orden creciente de sofisticación. Cada uno resuelve una limitación del anterior:

Autoencoder	Comprime una imagen a un código pequeño <i>z</i> y la reconstruye. Es la base: encoder, decoder y espacio latente.
VAE	Convierte ese latente en una <i>distribución</i> ordenada; por eso ya puede generar caras nuevas, no solo reconstruir.
GAN	Enfrenta dos redes (generador y juez). Gana nitidez a cambio de un entrenamiento inestable.
Diffusion	Aprende a quitar ruido paso a paso. Máxima calidad y diversidad, a cambio de un muestreo lento.

El método: realimentación agéntica

Construir el modelo es solo la mitad. La otra mitad es **mejorarlo con un bucle de realimentación**: agentes auxiliares proponen variantes (arquitecturas, hiperparámetros, técnicas), las **entrenan y miden** con métricas objetivas (PSNR, SSIM) sobre un conjunto de prueba, y nos quedamos con lo que de verdad mejora. En cada modelo verás el "antes" (la versión básica y sus problemas) y el "después" (la mejora encontrada y verificada).

A lo largo del cuaderno, las cifras (PSNR, SSIM, MSE, n° de parámetros) son **reales**, medidas sobre un conjunto de prueba que el modelo no usa para entrenar, y las capturas provienen directamente de la herramienta.

Autoencoder

Comprimir y reconstruir: el cuello de botella y el espacio latente

Teoría

Un autoencoder tiene forma de reloj de arena. El **encoder** reduce la imagen ($64 \times 64 \times 3 = 12.288$ números) a un vector pequeño z (el *código latente*); el **decoder** intenta reconstruir la imagen original solo a partir de z . Como z es mucho menor que la imagen, el modelo se ve obligado a quedarse con lo esencial.

$$x \rightarrow z = \text{encoder}(x) \rightarrow \hat{x} = \text{decoder}(z) \quad \text{pérdida} = \|x - \hat{x}\|^2 \text{ (MSE)}$$

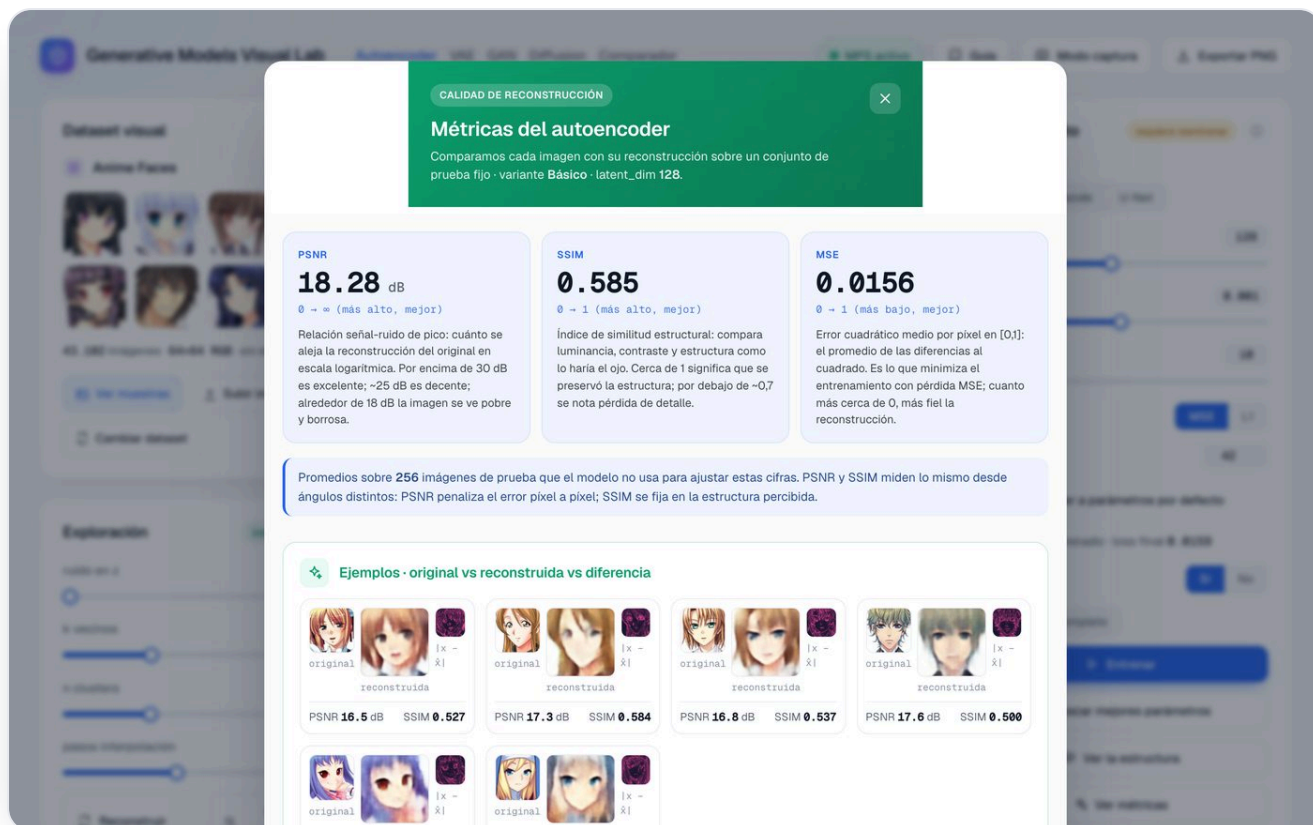
La clave pedagógica

Si z fuera tan grande como la imagen, el modelo podría limitarse a copiarla. El cuello de botella es lo que lo fuerza a *descubrir estructura*: que las caras tienen ojos, pelo, una pose... y a resumir todo eso en pocos números.

Parámetros importantes

PARÁMETRO	QUÉ CONTROLA
<code>latent_dim</code>	Tamaño de z . Por defecto 128 (compresión $\sim 96\times$). Pequeño $\rightarrow$ más abstracto y borroso; grande $\rightarrow$ más fiel, menos compresión.
<code>arquitectura</code>	Básico / Grande / U-Net. Define la capacidad y si hay <i>skip connections</i> (ver mejora).
<code>loss</code>	MSE (suaviza) o L1 (bordes algo más definidos).
<code>learning_rate</code> · <code>epochs</code>	Velocidad y duración del entrenamiento (con parada temprana).

La interfaz



Métricas de reconstrucción del modelo básico, medidas sobre 256 imágenes de prueba.

Problema → mejora agéntica

⚠ El problema

Con un cuello pequeño y pérdida MSE, la reconstrucción se difumina (la "maldición del promedio": ante la duda, el MSE elige el punto medio borroso). Además la red era pequeña. Resultado: **PSNR ~18 dB**, una imagen claramente suave.

✓ La mejora

Agentes auxiliares implementaron y **midieron** dos variantes: **Grande** (base 64 + bloques residuales, 10,7 M) y **U-Net** (skip connections, 5,0 M). Las skips dejan pasar el detalle fino que el cuello tiraba → salto grande en nitidez.

Comparación de variantes (medida real)

PSNR — relación señal/ruido (dB, más alto mejor)



Escala 0–30 dB. Por encima de 30 dB es excelente; ~18 dB es claramente borroso.

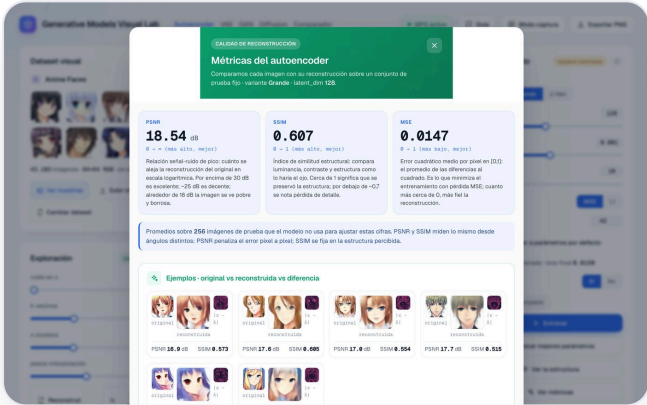
SSIM — similitud estructural (0–1, más alto mejor)



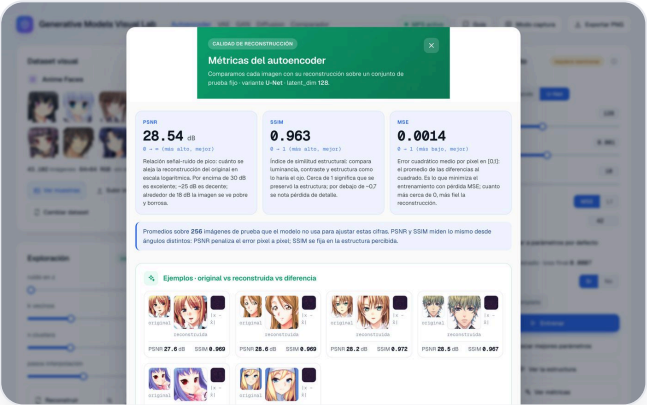
Variante	Parámetros	PSNR	SSIM	Lectura
Básico	2,43 M	18.28 dB	0.585	Cuello puro: borroso
Grande	10,72 M	18.54 dB	0.607	4× más grande... apenas mejora (sigue limitado por el cuello)
U-Net	5,05 M	28.54 dB	0.963	Las <i>skips</i> transforman la calidad con la mitad de parámetros que "Grande"

La lección que descubrió el bucle agéntico

Hacer la red **más grande no resolvió nada** (18,3 → 18,5 dB pese a 4× parámetros): el límite era el *cuello de botella*, no la capacidad. La mejora real fue **cambiar la arquitectura** a U-Net: +10 dB de PSNR y SSIM de 0,59 a 0,96, además con menos parámetros que la variante "Grande".



Variante Grande



Variante U-Net (skip connections)

Matiz de laboratorio: la U-Net reconstruye mejor porque sus *skips* evitan el cuello... pero por eso su *z* guarda menos información. Mejor reconstrucción no siempre significa mejor espacio latente: es justo el tipo de compromiso que el laboratorio deja ver.

VAE · Variational Autoencoder

De reconstruir a generar: un latente probabilístico y ordenado

Teoría

El VAE tiene el mismo esqueleto que el autoencoder, pero el encoder no produce un punto: produce una **distribución** (una media μ y una varianza σ^2). Se muestrea $z = \mu + \sigma \cdot \epsilon$ (reparametrización) y se añade un término **KL** que empuja el latente hacia una gaussiana estándar $N(0, I)$. Al ordenar así el espacio, basta con muestrear de esa gaussiana para **generar caras nuevas**.

$$\text{pérdida} = \text{reconstrucción} + \beta \cdot \text{KL}(N(\mu, \sigma^2) \parallel N(0, I))$$

Parámetros importantes

PARÁMETRO	QUÉ CONTROLA
β (peso del KL)	El compromiso clave. $\beta=0 \rightarrow$ un AE normal (latente desordenado); $\beta=1 \rightarrow$ equilibrio; β alto $\rightarrow$ latente muy ordenado pero muestras más borrosas.
latent_dim	Dimensión del latente probabilístico.
loss · lr · epochs	Como en el AE.

La interfaz y los resultados

Hereda el desenfoque del MSE y lo agrava la regularización KL: las caras generadas salen suaves y algo "promediadas".

Se expone **β como control interactivo** para explorar el compromiso, y el VAE reutiliza el backbone convolucional mejorado del autoencoder. La lección no es eliminar el desenfoque (es intrínseco), sino *entender de dónde viene* y por qué lo resuelven la GAN y la difusión.

Valor de β	Efecto en la reconstrucción	Efecto en el latente / generación
$\beta = 0$	Más nítida (es un AE)	Latente con huecos: muestrear da basura
$\beta = 1$	Algo más borrosa	Equilibrio: se puede generar y interpolar
$\beta > 1$	Más borrosa	Latente muy ordenado y suave

GAN · Generative Adversarial Network

Dos redes que compiten: nitidez a cambio de estabilidad

Teoría

La GAN tira el encoder y plantea un **juego entre dos redes**: el **generador** crea caras a partir de ruido z ; el **discriminador** intenta distinguir las reales de las falsas. Entrenan en competición. Como el "juez" es otra red (no un error por píxel), la GAN no tiene la presión hacia el promedio y suele dar **las caras más nítidas**.

Parámetros importantes

PARÁMETRO	QUÉ CONTROLA
<code>z_dim</code>	Tamaño del vector de ruido de entrada (100).
<code>learning_rate</code>	Crítico: si es muy alto, el juego se desequilibra y colapsa.
<code>epochs</code>	Iteraciones del juego adversarial.

La interfaz y los resultados

Generative Models Visual Lab

Autoencoder

VAE

GAN

Diffusion

Comparador

MPS activo

Guía

Modo captura

Exportar PNG

Galería generada

interactivo

Cada cara nace de un vector de ruido $z \sim N(0,1)$ que el generador convierte en imagen. Ninguna existe en el dataset.



nº de muestras

16

seed

42

Generar

Generar nuevas

Entrenamiento

requiere reentrenar

z_dim

100

learning_rate

0.0002

epochs

25

seed

42

Volver a parámetros por defecto

modelo entrenado · g_loss

5.5180 · d_loss

0.3794

Entrena para ver las curvas de pérdida de G y D en vivo.

Rápido

Completo

Entrenar

Ver la estructura

Interpolación en z

interactivo

Caminamos por el espacio de ruido entre dos puntos aleatorios. Como la GAN no tiene encoder, los extremos son ruido, no caras del dataset.

A

0.143

0.286

0.429

0.571

0.714

0.857

B



pasos

8

seed

7

Interpolación

Nueva interpolación

$$Z_\alpha = (1-\alpha) \cdot Z_A + \alpha \cdot Z_B$$

Pestaña GAN: galería de caras generadas desde ruido (ninguna existe en el dataset), interpolación en el espacio z y curvas de pérdida del generador y el discriminador.

Problema → mejora agéntica

⚠ El problema

El entrenamiento adversarial es inestable: con malas elecciones, el discriminador "gana" y el generador deja de aprender (g_loss se

✓ La mejora

Se aplicaron las recetas DCGAN verificadas: BatchNorm, LeakyReLU, Adam(0.5, 0.999), pérdida *non-saturating*, pesos iniciales $N(0, 0.02)$ y datos en $[-1, 1]$. Con

dispara), o aparece **colapso de modo** (todas las caras iguales).

esto el juego se equilibra y produce caras reconocibles.

~6,3 M de parámetros (generador + discriminador). La calidad es modesta a propósito (modelo pequeño, pocos epochs): el objetivo es *ver el mecanismo* adversarial, no competir con modelos enormes.

Diffusion · DDPM

Aprender a quitar ruido paso a paso — y un bug de muestreo revelador

Teoría

Un modelo de difusión aprende el proceso inverso de "ensuciar" una imagen. En el proceso **forward** se añade ruido gaussiano poco a poco hasta dejar ruido puro. En el **reverse**, una red (una UNet condicionada en el paso t) aprende a predecir y quitar ese ruido. Para generar, se parte de ruido puro y se limpia paso a paso hasta una cara.

Parámetros importantes

PARÁMETRO	QUÉ CONTROLA
timesteps (T)	Longitud del proceso de ruido (200). Es del modelo.
pasos de muestreo	Cuántos pasos se usan al generar. Más pasos = mejor, pero más lento.
schedule	Cómo se reparte el ruido (cosine por defecto, mejor que lineal a baja resolución).
learning_rate · epochs	Entrenamiento del denoising (pérdida MSE sobre el ruido predicho).

El problema: generaba ruido (y el modelo no tenía la culpa)

La primera versión **generaba manchas de color, no caras** — incluso al final del proceso. Lo desconcertante: la pérdida bajaba bien (de 0,50 a 0,084), señal de que la red *sí* estaba aprendiendo a quitar ruido.

⚠ La causa (depuración agéntica)

El muestreo aplicaba el paso DDPM clásico, que solo avanza de t a $t-1$, pero **submuestreando** (50 de 200 pasos).
Resultado: solo se quitaban 50 pasos de ruido de los 200 necesarios → la salida seguía siendo ruido. **El modelo estaba bien; el sampler estaba roto.**

✓ La solución

Se reescribió el muestreo a **DDIM** (determinista), que *salta correctamente* entre timesteps arbitrarios reusando la predicción de x_0 . Más **EMA de pesos** y **schedule cosine**.
Efecto inmediato: caras reconocibles — **y sin reentrenar**, porque el problema era solo cómo se muestreaba.

El resultado: de ruido a cara

Generative Models Visual Lab

Autoencoder

VAE

GAN

Diffusion

Comparador

MPS activo

Guía

Modo captura

Exportar PNG

Galería generada

desde ruido

Cada cara nace de ruido aleatorio puro y se «limpia» paso a paso hasta convertirse en un rostro nuevo. Ninguna existe en el dataset.

n° muestras

6

pasos de muestreo

50

seed

42

Generar nuevas

Nueva semilla

Proceso de difusión

la trayectoria

La misma imagen a lo largo del muestreo: de ruido puro (izquierda) a cara limpia (derecha). Cada instantánea quita un poco más de ruido.

ruido

t=200

t 175

7/50

t 146

14/50

t 118

21/50

t 85

29/50

t 57

36/50

t 28

43/50

cara

t=0

instantáneas

8

pasos de muestreo

50

seed

7

Nuevo proceso

Nueva semilla

Entrenamiento

requiere reentrenar

El modelo aprende a predecir el ruido que se añadió a una imagen. La pérdida es el error de esa predicción (MSE).

learning_rate

0.0002

epochs

30

timesteps (T)

200

seed

42

Volver a parámetros por defecto

modelo entrenado

loss final 0.0843

early stop

Si

No

Rápido

Completo

Entrenar

Ver la estructura

Entrenar no toca el checkpoint demo: el modelo entrenado vive durante esta sesión. La difusión es costosa, así que «Rápido» usa un subconjunto y pocas épocas.

Pestaña Diffusion tras el arreglo: la galería genera caras reconocibles y el panel "Proceso de difusión" muestra la trayectoria completa de **ruido puro** → **cara**.

Aspecto	Antes	Después
Muestreo	DDPM de un paso, submuestreado	DDIM (salta timesteps correctamente)
Pesos para muestrear	Crudos (oscilan con el minibatch)	EMA (media móvil, estables)
Resultado visual	Ruido de colores	Caras reconocibles
¿Reentrenar?	—	No: era solo el sampler

SÍNTESIS

Los cuatro, lado a lado

No hay un "mejor": hay compromisos

Generative Models Visual Lab

AutoencoderVAEGANDiffusionComparador

MPS activo

Guía

Modo captura

Exportar PNG

Tabla comparativa

PROPIEDAD	Autoencoder	VAE	GAN	Diffusion
¿Tiene encoder?	Sí	Sí	No	No
¿Genera caras nuevas?	No (reconstruye)	Sí (del prior)	Sí	Sí
Espacio latente	Determinista	Probabilístico $N(0, I)$	Ruido z	Ruido + tiempo t
Entrenamiento	Estable	Estable	Inestable (adversarial)	Estable
Muestreo	1 paso	1 paso	1 paso	Iterativo (lento)
Nitidez típica	Borrosa	Borrosa/suave	Nítida (con artefactos)	Buena

Galería comparada

seed 42Generar en todos

Autoencoder

Reconstrucción - no genera caras nuevas

VAE

Generadas - muestreo del prior

GAN

Generadas - desde ruido z

Diffusion

Generadas - denoising

Pestaña Comparador: tabla de capacidades y galería de los cuatro modelos con la misma semilla.

Propiedad	Autoencoder	VAE	GAN	Diffusion
¿Tiene encoder?	Sí	Sí	No	No

¿Genera caras nuevas?	No	Sí	Sí	Sí
Entrenamiento	Estable	Estable	Inestable	Estable
Muestreo	1 paso	1 paso	1 paso	Iterativo (lento)
Nitidez típica	Borrosa	Borrosa	Nítida	Buena

Conclusión

Cada modelo cambia un compromiso por otro: el VAE cede algo de nitidez por un latente ordenado; la GAN cambia estabilidad por nitidez; la difusión cambia velocidad por calidad. Y, en todos, el **bucle de realimentación agéntica** —proponer, entrenar, medir y quedarnos con lo que mejora— fue lo que convirtió un primer intento mediocre en algo que funciona: las variantes del autoencoder y, sobre todo, el arreglo del muestreo en difusión.

Hecho con un laboratorio 100% local. Código y guía de reproducción en el repositorio (licencia MIT).